

Euclidean

类欧几里德算法

定义

类欧几里德算法是洪华敦在 2016 年冬令营营员交流中提出的内容。

其本质可以理解为，使用一个类似辗转相除法的方法来进行函数求和。

引入

设

其中 c 是常数。需要一个 $O(\sqrt{n})$ 的算法。

这个式子和我们以前见过的式子都长得不太一样。带向下取整的式子容易让人想到数论分块，然而数论分块似乎不适用于这个求和。但是我们是可以做一些预处理的。

如果说 $j = \lfloor \frac{n}{k} \rfloor$ 或者 $k = \lfloor \frac{n}{j} \rfloor$ ，意味着可以将 k 对 j 取模以简化问题：

那么问题转化为了 $\sum_{k=1}^n \lfloor \frac{n}{k} \rfloor$ 的情况。观察式子，你发现只有 k 这一个变量。因此要推就只能从 k 下手。在推求和式子中有一个常见的技巧，就是条件与贡献的放缩与转化。具体地说，在原式 $\sum_{k=1}^n \lfloor \frac{n}{k} \rfloor$ 中， $k \leq \frac{n}{j}$ 是条件，而 j 是对总和的贡献。

要加快一个和式的计算过程，所有的方法都可以归约为 **贡献合并计算**。但你发现这个式子的贡献难以合并，怎么办？**将贡献与条件做转化** 得到另一个形式的和式。具体地，我们直接把原式的贡献变成条件：

现在多了一个变量 j ，既然算 k 的贡献不方便，我们就想办法算 j 的贡献。因此想办法搞一个和 j 有关的贡献式。这里有另一个家喻户晓的变换方法，笔者概括为限制转移。具体来说，在上面的和式中 $k \leq \frac{n}{j}$ 限制的上界，而 $j \leq \frac{n}{k}$ 限制的上界。为了搞 j ，就先把 j 放到贡献的式子里，于是我们交换一下 k 的求和算子，强制用 $k \leq \frac{n}{j}$ 限制的上界。

这样做的目的是让 k 摆脱 $k \leq \frac{n}{j}$ 的限制，现在 k 都被 $k \leq \frac{n}{j}$ 限制，而贡献式看上去是一个条件，但是我们仍把它叫作贡献式，再对贡献式做变换后就可以改变 k 的限制关系。于是我们做一些放缩的处理。首先把向下取整的符号拿掉

然后可以做一些变换

最后一步，向下取整得到：

这一步的重要意义在于，我们可以把变量 k 消掉了！具体地，令 $k = \frac{n}{j}$ ，那么原式化为

这是一个递归的式子。并且你发现 分子分母换了位置，又可以重复上述过程。先取模，再递归。这就是一个辗转相除的过程，这也是类欧几里德算法的得名。

容易发现时间复杂度为 $O(\log \min(a, b))$ 。

扩展

理解了最基础的类欧几里德算法，我们再来思考以下两个变种求和式：

推导 g

我们先考虑 $g(a, b)$ ，类似地，首先取模：

接下来考虑 $a \geq b$ 的情况，令 $a = kb + r$ 。之后的过程比较简略，因为方法和上文略同：

这时我们设 $g(a, b) = g_1(b, r)$ ，可以得到

推导 h

同样的，首先取模：

考虑 $a \geq b$ 的情况， $a = kb + r$ 。

我们发现这个平方不太好处理，于是可以这样把它拆成两部分：

这样做的意义在于，添加变量 k 的时候就只会变成一个求和算子，不会出现 k^2 的形式：

接下来考虑化简前一部分：

因此

在代码实现的时候，因为 g 和 h 两个函数各有交错递归，因此可以考虑三个一起整体递归，同步计算，否则有很多项会被多次计算。这样实现的复杂度是 $O(\log \min(a, b))$ 的。

实现

► [模板题代码实现](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[FFjet](#), [sshwy](#), [countercurrent-time](#), [cxm1024](#), [Early0v0](#), [Enter-tainer](#), [Great-designer](#), [H-J-Granger](#), [Henry-ZHR](#), [iamtwz](#), [lr1d](#), [ksyx](#), [megakite](#), [MegaOwler](#), [NachtgeistW](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [TonyYin0418](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用